Marking Scheme

Module Code: COMP I8062

Module Title: Algorithms

Internal Examiner(s): Dr. Michelle Graham

Session: Winter

Year: 25-26

Instructions:

Answer QUESTION ONE (40 marks) and any other TWO questions (30 marks each).

Marking Scheme

Marking Scheme

You MUST attempt this question.

Question 1) Algorithm Design and Complexity

(a) **Time complexity** and **space complexity** are two key metrics for the analysis of algorithms.

(i) Define time complexity and space complexity in the context of algorithm analysis. Explain why these measures are important when comparing or evaluating algorithms.

(5 marks)

KEY ASPECTS:

Time complexity: Time required for an algorithm to complete relative to the input size. Usually expressed in Big O (growth rate of algorithm runtime relative to input size)

Space complexity: Space required for an algorithm to complete relative to the input size. Accounts for memory used by variables, data structures, recursion stacks etc

Importance: Predicts performance, allows for comparison, indicates appropriate resource allocation requirements, indicates scalability. Essentially allows for informed decision-making.

MARKS BREAKDOWN:

Definition: 1 Mark * 2

Importance: 3 marks

(ii) Explain the difference between **best-case**, **worst-case**, and **average-case** time complexities. Provide an example of an algorithm where the best-case and worst-case complexities differ significantly.

(5 marks)

MARKS BREAKDOWN:

Definition: 1 Mark * 3

Example: 2 marks (E.g. Quicksort best of O(n log n) vs worst of $O(n^2)$)

Marking Scheme

(iii) Outline two **additional considerations** that can be used to assess algorithms beyond worst-case time complexity. Explain what each adds to the evaluation of an algorithm's performance.

(8 marks)

KEY ASPECTS:

Possible considerations:

- Scalability
- Memory efficiency
- Stability
- Ease of implementation
- Adaptability to data
- Average performance consistency
- Real world execution time (performance metrics on real hardware)

MARKS BREAKDOWN:

1 mark * 2 - Naming consideration

3 marks * 2 - What this consideration adds to evaluation

(b) The algorithm found in appendix A is an implementation of Binary Search.

(i) Write and explain the **code** for an alternative implementation of Binary Search that employs recursion instead of iteration. Indicate and explain the base case and recursive case in your implementation

(16 marks)

Marking Scheme

KEY ASPECTS:

```
public static int search(int [] data, int value) { return search(data, value, left: 0,
    right: data.length-1); }

private static int search(int [] data, int value, int left, int right){
    if(left > right){
        return -1;
    }

    int mid = left + (right-left)/2;
    if(data[mid] == value){
        return mid;
    }else if(data[mid] < value){
        left = mid+1;
    }else{
        right = mid-1;
    }

    return search(data, value, left, right);
}
```

Base case: left > right **AND** data[mid] == value

Recursive case: search(data, value, left, right)

MARKS BREAKDOWN:

Code:	5 marks
Explanation:	5 marks
Base case & explanation:	3 marks
Recursive case & explanation:	3 marks

(ii) Does the **space** complexity of your implementation of Binary Search differ from that of the original implementation shown in appendix A? Justify your answer with reference to the space complexity of each implementation.

(6 marks)

KEY ASPECTS:

Yes. Original is O(1) as it requires a constant amount of memory to process all input. New approach is O(log n) as it has to recurse log n times to process all input, incurring log n recursive stack calls.

Marking Sc

[If their solution is not correct, this question should be marked based on how the student implemented the solution]

MARKS BREAKDOWN:

Verdict:	1 mark
Giving space complexities:	2 marks
Explanations:	2 marks

Choose any TWO questions from questions 2, 3 and 4

Question 2) Sorting Algorithms

(a) **Bubble sort** and **Quick sort** are examples of sorting algorithms using different algorithmic approaches.

(i) Compare the time complexities of these algorithms. Explain under what (if any) conditions one of these algorithms might perform better than the other.

(5 marks)

SAMPLE ANSWER:

Time complexities: Both will incur $O(n^2)$ in worst case, but Bubble sort improves to O(n) for sorted data when optimisation (tracking if swaps have occurred in a pass) is included, while Quick sort has average performance of O(n log n)

MARKS BREAKDOWN:

Time complexities:	1 mark * 2
Bubble sort improvement:	3 marks

(ii) What is stability in sorting? Give an example where stability is required.

(5 marks)

SAMPLE ANSWER:

Marking Scheme

“Stability in Sorting refers to a property of sorting algorithms where elements with equal keys (or values) maintain their relative order after sorting. In other words, if two items are equal in terms of the sorting key, a stable sort will ensure that they appear in the same order in the sorted output as they did in the original input.”

Example requiring stability: Multi-level/multi-attribute sorting

MARKS BREAKDOWN:

Explanation: 3 marks

Example: 2 marks

(iii) Only one of these algorithms is stable by default. Identify which algorithm is stable, explaining why the stability property applies to one but not the other.

(5 marks)

SAMPLE ANSWER:

Bubble sort is stable as it will not move equal values. quick sort may shuffle values while moving elements around the pivot.

MARKS BREAKDOWN:

Verdict: 2 marks

Explanation: 3 marks

(b) **Bucket Sort** is a sorting algorithm that distributes elements of an array into several "buckets" and then sorts each bucket individually, typically using another sorting algorithm. Discuss an appropriate sort algorithm to be used on these individual buckets for each of the following considerations:

(i) Small bucket sizes are guaranteed.

(ii) Sort stability is required.

(iii) Data is not uniformly distributed across buckets.

(15 marks)

Marking Scheme

SAMPLE ANSWER:

Small bucket sizes: simple/naïve algorithm for fast development and deployment, or quicksort for fast sorting without high memory cost

Sort stability: bubble sort or merge sort (merge sort where memory costs are not a consideration or dataset will be large)

Unpredictable data size: Sort algorithm that will perform efficiently no matter the size (merge or quick)

MARKS BREAKDOWN:

Verdict: 1 mark * 3

Explanation: 4 marks * 3

Question 3) Two-Pointer Technique

(a) Explain the **two-pointer technique**. In what types of problems is this technique commonly used, and why is it effective?

(6 marks)

SAMPLE ANSWER:

Two-pointer technique: A common algorithmic strategy used to solve problems efficiently by using two pointers (or indices) that move through a data structure, often an array or list, in a coordinated manner.

How the Two-Pointer Technique Works

The two-pointer approach typically involves:

- **Initializing two pointers** at different positions, often at the beginning and end of an array, or both starting at the beginning.
- **Moving the pointers** toward each other or in specific directions based on certain conditions to achieve the desired outcome.

Types of problem:

- Palindrome checks
- Reversing
- Finding pairs
- Finding groupings

Marking Scheme

- Removing/partitioning

Reason for effectiveness:

- Reduces time complexity
- Operates in constant space

MARKS BREAKDOWN:

Explanation:	2 marks
Problem types:	1 mark * 2
Reasons for efficacy:	1 mark * 2

(b) The algorithm shown in **appendix B** finds the maximum sum that can be generated from a specified-size sequence of values in an array of integers.

(i) How is the sliding-window implementation of the two-pointer technique being used here? Why does this approach remove the need to re-sum the elements of each subarray from scratch?

(6 marks)

SAMPLE ANSWER:

Two indices define the current "window." (starting at numElements and 0 respectively)

As the window slides, one pointer moves ("slides") forward to add in the new value, while the other effectively drops the oldest value by subtracting it from the total.

Reusing previous sums avoids O(k) re-computation per subarray.

Leads to O(n) time complexity in comparison to O(n*k)

MARKS BREAKDOWN:

Explanation:	4 marks
Need for sorted data:	2 marks

Marking Scheme

(ii) Write and explain the **pseudocode** for a *brute-force* solution to this problem (finding the maximum sum of any contiguous subarray of a specified length).

(10 marks)

PSEUDOCODE:

Inputs: Array to locate maximum sum in a sequence of array slots (data), size of sequence to consider (numElements)

Output: Largest possible sum found for any sequence of array slots of size numElements

Create maxSum to store smallest possible number (Integer's minimum value)

for each position in data (i) from 0 to array length - numElements + 1:

    Create currentSum to store sum of current window in array

    from 0 to numElements (j):

        Add data[i+j] to windowSum

    if windowSum is greater than maxSum:

        Set maxSum to windowSum

Return maxSum

MARKS BREAKDOWN:

Pseudocode: 5 marks

Explanation: 5 marks

(iii) Compare and contrast the time complexity of your brute-force implementation to that of the original two-pointer implementation. In what situations, if any, might the brute-force approach be preferable? Justify your answer.

(8 marks)

Marking Scheme

SAMPLE ANSWER:

Time Complexity Comparison

- **Brute-Force Approach**: The brute-force solution has a time complexity of O(n*k), where n is the number of elements in the array and k is the window size. This is because it checks every sequence of k elements, and these overlap.
- **Two-Pointer Approach**: The two-pointer solution, on the other hand, has a time complexity of O(n) as it does not need to recompute sub-sequences. Instead, each time it starts a new sequence, it removes the element before the sequence and adds the new element joining the sequence. This avoids redoing the calculation for the overlapped elements, resulting in linear time complexity.

Thus, the two-pointer approach is significantly more efficient than the brute-force approach when dealing with larger arrays or where the number of elements to be included in the sequence is high relative to the size of the array, especially for time-sensitive applications or larger datasets.

Situations Where Brute-Force May Be Preferred

- Small datasets
- Low number of elements to be included relative to the size of the array
- Prototyping/clarity of implementation

MARKS BREAKDOWN:

Comparison and explanation of time complexities:	4 marks
Naming situation in which brute force makes sense:	2 marks
Justification:	2 marks

Question 4) Selecting Algorithms

(a) You are processing an IoT dataset collected as an array of timestamped events. Due to a sensor malfunction, the data is now unsorted and contains duplicate entries. Rather than discarding the dataset entirely, you have chosen to try and filter out the duplicate information before continuing with the remainder of your processing. This is envisioned as a standalone action, i.e. it does not need to be repeated.

Marking Scheme

(i) Outline an approach to deduplicating unordered data. What is the time complexity of this process?

(6 marks)

SAMPLE ANSWER:

Explain standard array-based deduplication, employing a hashSet to filter [or any other logical solution is acceptable]

MARKS BREAKDOWN:

Explanation: 4 marks

Time complexity: 2 marks (O(n) for hashset assuming constant insertion, $O(n^2)$ for nested loop approach)

(ii) Deduplicating sorted data may perform with better time complexity than deduplicating unsorted data. Outline an approach to deduplicating sorted data and state its time complexity. Does this approach produce better time complexity than your outlined approach from part (i)? Explain why this is the case.

(12 marks)

SAMPLE ANSWER:

Array-based deduplication with reference to the last populated slot in the array (may also discuss a 2 pointer approach tracking the extent of a "run" of data and only storing when the values at either end of the window differ) (O(n))

MARKS BREAKDOWN:

Approach explanation:	6 marks
Time complexity:	2 marks
Better/same/worse?:	1 mark
Explanation:	3 marks

Marking Scheme

(iii) A teammate has suggested sorting the data before carrying out deduplication to allow the use of the sorted deduplication algorithm. Write a short note discussing the comparative performance of taking this approach vs deduplicating the original (unordered) array and recommending an approach. Your discussion should specify the sorting algorithm that would be employed as well as any other relevant details.

(12 marks)

SAMPLE ANSWER:

Compare array-based sorted deduplication to original outlined approach. Expecting mention of high-performance stable sort algorithm (merge or tim), highlighting importance of stability and with acknowledgement of space considerations if choosing/not choosing merge.

Any overall verdict is acceptable if it has been appropriately justified with relevant information and consideration.

MARKS BREAKDOWN:

Discussion: 6 marks

Verdict: 2 marks

Justification: 4 marks

Marking Scheme

APPENDIX A:

```
349     public static int search(int [] nums, int value){
350         if(nums == null){
351             throw new IllegalArgumentException("Array to be searched cannot be null");
352         }
353
354         int start = 0;
355         int end = nums.length-1;
356
357         while(start <= end) {
358             int mid = start + (end - start) / 2;
359
360             if (nums[mid] == value){
361                 return mid;
362             }else if(value > nums[mid]){
363                 start = mid + 1;
364             }else{
365                 end = mid -1;
366             }
367
368         }
369         return -1;
370     }
```

Marking -

APPENDIX B:

```java
public static int maxSum(int arr[], int numElements){
    if(arr == null || arr.length == 0){
        throw new IllegalArgumentException("Cannot find max sum in null or empty array");
    }

    if(numElements > arr.length){
        throw new IllegalArgumentException("Number of elements to be included in sum must not
            exceed length of array");
    }

    int maxSum = 0;
    for (int i = 0; i < numElements; i++) {
        maxSum += arr[i];
    }

    int windowSum = maxSum;
    for (int i = numElements; i < arr.length; i++) {
        windowSum += arr[i];
        windowSum -= arr[i-numElements];

        maxSum = Math.max(windowSum, maxSum);
    }

    return maxSum;
}
```

Marking